

OMGRec: One-time Matching-based Generative Rerank with Permutation-level Modeling in E-commerce

Junwei Xu*
Shenzhen Key Laboratory of
Ubiquitous Data Enabling, Shenzhen
International Graduate School
Tsinghua University
Shenzhen, China
xjw23@mails.tsinghua.edu.cn

Zhibo Xiao
Chuxin Chen
xiaozhibo.xzb@alibaba-inc.com
chenchuxin.ccx@alibaba-inc.com
Taobao & Tmall Group of Alibaba
Hangzhou, China

Chengyu Lai
Qijie Shen
lcy477544@alibaba-inc.com
qijie.sqj@alibaba-inc.com
Taobao & Tmall Group of Alibaba
Hangzhou, China

Jiuning Lin
Dimin Wang
linjiuning.ljn@alibaba-inc.com
dimin.wdm@alibaba-inc.com
Taobao & Tmall Group of Alibaba
Hangzhou, China

Jialin Zhu
xiaofei.zjl@alibaba-inc.com
Taobao & Tmall Group of Alibaba
Hangzhou, China

Xiao-Ping Zhang[†]
Shenzhen Key Laboratory of
Ubiquitous Data Enabling, Shenzhen
International Graduate School
Tsinghua University
Shenzhen, China
xpzhang@ieee.org

Abstract

Reranking in large-scale e-commerce recommender systems determines the final exposure list by modeling user intent with contextual interactions among candidate items. However, most existing approaches either rely on point-wise scoring, which overlooks permutation-level dependencies, or adopt autoregressive paradigms that enhance context modeling at the cost of prohibitively high inference latency in practical applications. To address these limitations, we present **OMGRec**, a one-time matching-based generative reranking method. OMGRec employs request-level modeling with a lightweight dual-encoder architecture: a candidate encoder and a position encoder jointly produce a position-wise allocation matrix. Furthermore, a *candidate-aware interest fuser* dynamically enriches the user representation by attending to heterogeneous behavioral sequences and contextual candidate information, and a *permutation perception module* leverages masked prediction to explicitly capture permutation-level cues. To prioritize business-critical metrics, we introduce a sample-wise importance weighting scheme driven by user browse depth, clicks, and conversions.

Extensive offline evaluations demonstrate that OMGRec consistently outperforms the baselines, improving HR@20 by +4.27% and NDCG@20 by +2.13%. Large-scale online A/B tests on the Taobao App further show a +0.34% lift in Click, along with +0.39% and +1.88% improvements in Order and GMV, respectively.

CCS Concepts

• Information systems → Retrieval models and ranking.

*This work is done during an internship at Alibaba.

[†] Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792875>

Keywords

Recommender System, Re-ranking, Generative Models

ACM Reference Format:

Junwei Xu, Zhibo Xiao, Chuxin Chen, Chengyu Lai, Qijie Shen, Jiuning Lin, Dimin Wang, Jialin Zhu, and Xiao-Ping Zhang. 2026. OMGRec: One-time Matching-based Generative Rerank with Permutation-level Modeling in E-commerce. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3774904.3792875>

1 Introduction

Reranking is the final stage of industrial cascaded recommender systems, transforming a scored candidate set into the exposure list that users actually see. Beyond assessing item relevance, effective reranking must reason about permutation-level dependencies [7]. The combinatorial nature of the reranking task exacerbates both modeling and efficiency challenges. Classic point-wise methods [2, 3] capture local relevance well, but typically ignore cross-item context and position effects, which limits list-wise quality. Specifically, the key business metrics such as CTR and CVR vary across different positions [12]. Point-wise rerankers with position-invariant scores cannot model candidates–position interactions, struggling to optimize list-wise utility that depends jointly on which candidates are shown and where they appear.

A growing body of work frames reranking as sequence generation, often with a generator–evaluator paradigm: a generator proposes permutations and an evaluator estimates their list-wise utility [4, 8, 10]. For generators, although autoregressive (AR) generation is well-suited for capturing sequential dependencies, it is prone to exposure bias and error propagation, and its sequential decoding induces unacceptable latency [9]. Non-autoregressive (NAR) alternatives, such as NLGR [11] and DCDR [5], generate sequences in a faster way, but they still rely on iterative refinement steps with uncertain convergence to enhance overall permutation quality.

We propose **OMGRec**, a one-time matching-based generative reranking model for e-commerce recommendation. OMGRec achieves

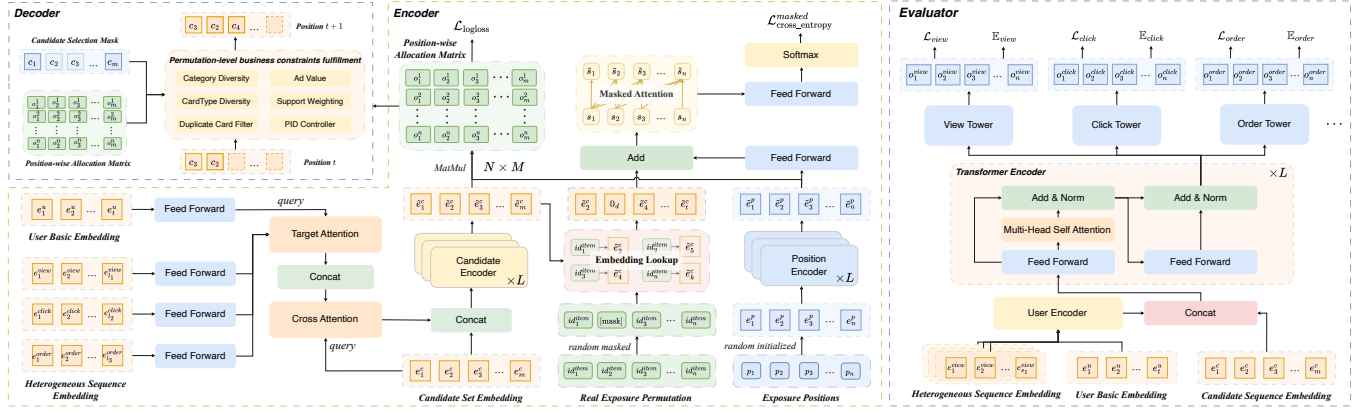


Figure 1: Left part shows the overall architecture of OMGRec. Right part illustrates our evaluator model.

one-time inference through a dual-encoder architecture that simultaneously processes candidates and positions, producing a single allocation matrix without iterative refinement. To strengthen user representation, a *candidate-aware interest fuser* (CIF) dynamically attends to heterogeneous behavioral sequences, aligning user intent with the current candidate set. To inject list-level signals without multiple inference passes, a *permutation perception module* (PPM) employs masked sequence modeling to learn global permutation cues. Finally, we introduce sample-wise importance weights that prioritize high-quality training sequences to better align optimization with business-critical objectives. Both offline and online experiments on the Taobao App, one of the world's largest e-commerce platforms, confirm the effectiveness and practicality of our method.

2 Problem Formulation

Reranking operates on an *unordered candidate set* C produced by the upstream ranking stage, aiming to select and arrange a subset of candidates into an *ordered exposure list* $\mathcal{V}$ that maximizes expected user engagement. Formally, given a user set $\mathcal{U}$ and an item set $\mathcal{I}$, the request-level reranking sample is represented as $(u, C, \mathcal{V}, \mathcal{Y})$, where $u \in \mathcal{U}$ is the target user, $C \subset \mathcal{I}$ is the candidate set with $|C| = m$, $\mathcal{V} = [v_1, v_2, \dots, v_n]$ is the ordered exposure sequence with $n \leq m$, and $\mathcal{Y} = \{y_{uv}^t\}$ denotes user feedback. We define $y_{uv}^t = 1$ if user u interacts with item v_t at position t (e.g., click, purchase).

The reranking task is to learn a mapping function $f : (u, C) \rightarrow \mathcal{V}$ that generates the optimal exposure sequence $\mathcal{V}^*$. Unlike point-wise rankers that score items independently, reranking must capture cross-candidate dependencies to optimize list-wise utility.

3 Methodology

Figure 1 illustrates the overall architecture of OMGRec. We will elaborate on each component below.

3.1 Candidate-aware Interest Fuser

User understanding is critical for personalized reranking; yet, it remains challenging to align heterogeneous behavioral histories with the current candidate set. To address this, we design a Candidate-aware Interest Fuser (CIF) to dynamically generate user representations aligned with each candidate.

Specifically, we first obtain the basic user embedding from the user profiles. Heterogeneous behavior sequences (e.g., browsing, adding-to-cart, and purchasing) are encoded and fused with target-aware attention conditioned on the basic embedding. We then concatenate the outputs with the basic embedding to form a comprehensive user context. To align with the candidate set, we embed all candidates and apply cross-attention with candidates as queries and the user context as key-value to produce candidate-aware user signals. The resulting representation is broadcast and concatenated with each candidate embedding for subsequent encoding.

3.2 Main Encoder

Our main encoder decouples candidate and position modeling, enabling efficient computation and fine-grained position-awareness.

For each candidate, we concatenate its embedding with the candidate-aware user representation $\mathbf{e}^u$. The candidate encoder then applies L layers of Transformer blocks to capture inter-item dependencies. Notably, we incorporate user-aware attention into the candidate encoder by computing the Hadamard product between the output of the vanilla attention and the transformed user representation $\phi(\mathbf{e}^u)$:

$$\text{Attention}^*(Q, K, V, \mathbf{e}^u) = \left(\text{softmax} \left(\frac{QK^\top}{\sqrt{d}} \right) V \right) \odot \phi(\mathbf{e}^u). \quad (1)$$

Similarly, to model position-specific characteristics, we initialize learnable position embeddings $\{\mathbf{e}_1^p, \mathbf{e}_2^p, \dots, \mathbf{e}_n^p\}$, where $\mathbf{e}_j^p \in \mathbb{R}^d$ encodes the properties of the j -th display position. These embeddings are processed by another L -layer Transformer.

Suppose the encoded candidate and position representations are $\tilde{\mathbf{E}}_c \in \mathbb{R}^{m \times d}$ and $\tilde{\mathbf{E}}_p \in \mathbb{R}^{n \times d}$, we then compute the position-wise allocation matrix via scaled dot-product, which captures the compatibility between each candidate and each position:

$$\hat{\mathbf{O}} = \text{softmax} \left(\frac{\tilde{\mathbf{E}}_p \tilde{\mathbf{E}}_c^\top}{\sqrt{d}} \right) \in \mathbb{R}^{n \times m}, \quad (2)$$

where $\hat{o}_{ij}^i$ is the probability of assigning candidate j to position i .

3.3 Permutation Perception Module

While the allocation matrix provides position-wise scores, it does not explicitly model the sequential dependencies and permutation-level patterns inherent in real user exposure sequences. To address this, we introduce a Permutation Perception Module (PPM) that leverages masked sequence modeling to inject permutation-level signals into the training process.

Given the ground-truth exposure sequence $\mathcal{V} = [v_1, v_2, \dots, v_n]$, we randomly mask a subset of positions with probability p_{mask} , replacing their embeddings with a mask token $\mathbf{e}_{[\text{MASK}]}$. Positional encodings are then added to obtain position-aware sequence embeddings, after which a masked attention is applied to model the contextual dependencies among exposed items. For each masked position i , we predict the original item via a classification head and compute the cross-entropy loss $\mathcal{L}_{\text{perm}}$. This auxiliary task encourages the model to capture permutation-level context, complementing the main allocation objective.

3.4 Training Strategy

The primary objective is to align the predicted allocation matrix $\hat{\mathbf{O}} \in \mathbb{R}^{n \times m}$ with the ground-truth exposure assignment $\mathbf{O} \in \mathbb{R}^{n \times m}$, where $o_j^i = 1$ if candidate j is exposed at position i , and $o_j^i = 0$ otherwise. We use cross-entropy loss:

$$\mathcal{L}_{\text{main}} = - \sum_{i=1}^n \sum_{j=1}^m o_j^i \log(\hat{o}_j^i). \quad (3)$$

User engagement quality varies significantly across requests: some users browse deeply and have transaction behavior, while others abandon quickly. To prioritize high-quality training signals, we introduce a sample-wise importance weight w for each request:

$$w = \alpha + \beta \cdot \frac{d_{\text{view}}}{n}, \quad (4)$$

where d_{view} is the browse depth (number of items viewed), n is the total sequence length, and α, β are hyperparameters. This weight can be further enriched with click and conversion signals, e.g., $w = \alpha \cdot w_{\text{view}} + \beta \cdot w_{\text{click}} + \gamma \cdot w_{\text{order}}$. The final training loss can be formulated as:

$$\mathcal{L} = w (\mathcal{L}_{\text{main}} + \lambda \mathcal{L}_{\text{perm}}), \quad (5)$$

where λ is a balancing hyperparameter.

3.5 Decoder and Evaluator

At inference time, we decode the final exposure sequence $\mathcal{V}$ from the allocation matrix $\hat{\mathbf{O}} \in \mathbb{R}^{m \times n}$ via greedy selection. For each position $i = 1, 2, \dots, n$, we perform the following selection:

$$v^i = \arg \max_{c_j \in C' \cap \mathcal{F}^i} \hat{o}_j^i, \quad (6)$$

where C' is the set of remaining candidates, and $\mathcal{F}^i$ represents those that satisfy position-specific business constraints (e.g., category diversity, advertising). Once v^i is selected, it is removed from C' for subsequent positions. Greedy decoding can be replaced with beam search to explore more flexible generation.

After generating multiple sequences, we employ an evaluator that estimates list-wise utility regarding multiple business objectives. As shown in Figure 1, the evaluator encodes the user and the

generated sequence through embedding and Transformer layers, then predicts various metrics via a multi-task learning framework.

For the view task, we model the probability that the user browses to position i or beyond, given that position $i - 1$ was viewed. Let o_i^{view} denote this conditional probability. The probability of viewing exactly i items and the expected browse depth are computed as:

$$P_{\text{view}}(i) = P_{\text{view}}(\geq i) - P_{\text{view}}(\geq i + 1) = \prod_{j=1}^i o_j^{\text{view}} - \prod_{j=1}^{i+1} o_j^{\text{view}}, \quad (7)$$

$$\mathbb{E}_{\text{view}} = \sum_{i=1}^n i \cdot P_{\text{view}}(i) = \sum_{i=1}^n \prod_{j=1}^i o_j^{\text{view}}. \quad (8)$$

Similarly, we compute expectations for other objectives like clicks and conversions, represented as $\mathbb{E}_{\text{click}}$ and $\mathbb{E}_{\text{order}}$. The overall list-wise utility score is a weighted combination:

$$\mathcal{V} = \gamma_1 \mathbb{E}_{\text{view}} + \gamma_2 \mathbb{E}_{\text{click}} + \gamma_3 \mathbb{E}_{\text{order}} + \dots, \quad (9)$$

where $\{\gamma_k\}$ are task-specific weights reflecting business priorities. The sequence with the highest utility score will then be validated for final exposure.

4 Experiments

4.1 Experimental Setup

4.1.1 Dataset. We conduct experiments on real-world traffic logs collected from Taobao's homepage feed. Each reranking request is aggregated with comprehensive contextual information, including user profile features, heterogeneous behavioral sequences, the candidate item set and the final exposure sequence, along with their corresponding feedback signals. The dataset spans a 7-day period in September 2025, with the first six days used for training and the final day for evaluation. Each day contains approximately 0.8 Billion reranking requests. For each request, the reranking model needs to select and order *dozens* of items from *hundreds* of candidates.

4.1.2 Baseline. We compare OMGRec against several competitive reranking methods: DNN, DeepFM [3], SetRank [6], Seq2Slate [1], NAR4Rec [8], and NLGR [11]. These baselines cover a range of architectures, including point-wise scoring models, set-based models, and generative models. Note that we exclude methods like PIER [10] that are designed for small-scale scenarios (e.g., selecting 3 items from 10 candidates), not applicable to our large-scale setting.

4.1.3 Evaluation Metrics. We evaluate the performance of different reranking methods using standard ranking metrics that assess both relevance and position-awareness. Specifically, we primarily report HR@20 and NDCG@20. In our context, HR = 1 when items are predicted in the actual exposed positions. For HR@20, we will separately list the metrics for those samples with at least one click or order in the top-20 positions, denoted as HR^{click}@20 and HR^{order}@20, respectively. For NDCG@20, we consider exposure, click and order relevance signals to compute the metric.

4.1.4 Implementation Details. Both OMGRec and baseline models are implemented using TensorFlow 1.12.0 and trained on a cluster of NVIDIA H20 GPUs. For all models, we use the improved AdamDecay optimizer with a batch size of 1024. The learning rate is initially set to 0.01 and will be linearly decayed to 0.001.

Table 1: Offline performance comparison results. Best results are in bold and second-best are underlined.

Methods	DNN	DeepFM	SetRank	Seq2Slate	NAR4Rec	NLGR	w/o PPM	w/o CIF	w/o w	OMGRec
NDCG@20	0.4552	0.4783	0.5025	0.5092	0.5257	0.5298	0.5280	0.5350	<u>0.5384</u>	0.5411
HR@20	0.1505	0.1648	0.1835	0.1902	0.1984	0.2013	0.2038	0.2078	<u>0.2093</u>	0.2099
HR@20 ^{clk}	0.1176	0.1463	0.1548	0.1613	0.1698	0.1727	0.1753	<u>0.1800</u>	0.1799	0.1828
HR@20 ^{order}	0.1162	0.1423	0.1540	0.1585	0.1670	0.1694	0.1687	0.1690	<u>0.1734</u>	0.1746

Table 2: Online A/B testing on Taobao’s homepage feed.

	Click	Order	Exposure	GMV	Failover
OMGRec	+0.3376%	+0.3941%	+0.3110%	+1.8825%	+0.01%

4.2 Offline Experiments

Table 1 summarizes the overall performance of OMGRec and baseline methods. We observe that OMGRec consistently outperforms all baselines across various metrics, demonstrating its effectiveness. It validates the advantages of our one-time matching-based generative reranking approach with permutation-level modeling.

To assess the contributions of key components in OMGRec, we conduct an ablation study by removing specific modules: (1) **w/o CIF** removes the Candidate-aware Interest Fuser; (2) **w/o PPM** removes the Permutation Perception Module; (3) **w/o w** removes weighted training. Results show that CIF improves user interest modeling by integrating candidate context, and PPM has a particularly significant impact on permutation-level modeling, reflecting on NDCG@20. Weighted training also contributes to better alignment with business objectives, enhancing overall performance.

4.3 Online A/B Testing

We deploy OMGRec in Taobao’s homepage feed and conduct a large-scale online A/B test. The results can be seen in Table 2. While maintaining comparable failover rates, OMGRec demonstrates substantial improvements across all core business metrics compared to the production baseline, a highly-optimized sequential reranking model. These results further confirm the practical effectiveness of OMGRec in a real-world e-commerce setting.

5 Conclusion

We presented OMGRec, a one-time matching-based generative reranking method that addresses the fundamental trade-off between permutation-level modeling and online efficiency in large-scale e-commerce recommendation. Extensive experiments on the Taobao App demonstrate that OMGRec consistently outperforms existing baselines in both offline metrics and online A/B testing. Future work may explore advanced methods for permutation-level information injection as well as reinforcement learning.

Acknowledgments

This research is supported by the Shenzhen Ubiquitous Data Enabling Key Lab under grant ZDSYS20220527171406015 and by the Tsinghua Shenzhen International Graduate School-Shenzhen Pengrui Endowed Professorship Scheme of the Shenzhen Pengrui Foundation.

References

- [1] Irwan Bello, Sayali Kulkarni, Sagar Jain, Craig Boutilier, Ed Chi, Elad Eban, Xiyang Luo, Alan Mackey, and Ofer Meshi. 2019. Seq2slate: Re-ranking and slate optimization with rnn. *ICLR* (2019).
- [2] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishu Aradhya, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Isipir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. 2016. Wide & Deep Learning for Recommender Systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems* (Boston, MA, USA) (DLRS 2016). Association for Computing Machinery, New York, NY, USA, 7–10. doi:10.1145/2988450.2988454
- [3] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: a factorization-machine based neural network for CTR prediction. *arXiv preprint arXiv:1703.04247* (2017).
- [4] Guangda Huzhang, Zhen-Jia Pang, Yongqing Gao, Yawen Liu, Weijie Shen, Wen-Ji Zhou, Qianying Lin, Qing Da, An-Xiang Zeng, Han Yu, Yang Yu, and Zhi-Hua Zhou. 2023. AliExpress Learning-to-Rank: Maximizing Online Model Performance Without Going Online. *IEEE Transactions on Knowledge and Data Engineering* 35, 2 (2023), 1214–1226. doi:10.1109/TKDE.2021.3098898
- [5] Xiao Lin, Xiaokai Chen, Chenyang Wang, Hantao Shu, Linfeng Song, Biao Li, and Peng Jiang. 2024. Discrete Conditional Diffusion for Reranking in Recommendation. In *Companion Proceedings of the ACM Web Conference 2024* (Singapore, Singapore) (WWW '24). Association for Computing Machinery, New York, NY, USA, 161–169. doi:10.1145/3589335.3648313
- [6] Liang Pang, Jun Xu, Qingyao Ai, Yanyan Lan, Xueqi Cheng, and Jirong Wen. 2020. SetRank: Learning a Permutation-Invariant Ranking Model for Information Retrieval. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval* (Virtual Event, China) (SIGIR '20). Association for Computing Machinery, New York, NY, USA, 499–508. doi:10.1145/3397271.3401104
- [7] Changhua Pei, Yi Zhang, Yongfeng Zhang, Fei Sun, Xiao Lin, Hanxiao Sun, Jian Wu, Peng Jiang, Junfeng Ge, Wenwu Ou, and Dan Pei. 2019. Personalized re-ranking for recommendation. In *Proceedings of the 13th ACM Conference on Recommender Systems* (Copenhagen, Denmark) (RecSys '19). Association for Computing Machinery, New York, NY, USA, 3–11. doi:10.1145/3298689.3347000
- [8] Yuxin Ren, Qiya Yang, Yichun Wu, Wei Xu, Yalong Wang, and Zhiqiang Zhang. 2024. Non-autoregressive Generative Models for Reranking Recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Barcelona, Spain) (KDD '24). Association for Computing Machinery, New York, NY, USA, 5625–5634. doi:10.1145/3637528.3671645
- [9] Florian Schmidt. 2019. Generalization in Generation: A closer look at Exposure Bias. In *Proceedings of the 3rd Workshop on Neural Generation and Translation*, Alexandra Birch, Andrew Finch, Hiroaki Hayashi, Ioannis Konstas, Thang Luong, Graham Neubig, Yusuke Oda, and Katsuhito Sudoh (Eds.). Association for Computational Linguistics, Hong Kong, 157–167. doi:10.18653/v1/D19-5616
- [10] Xiaowen Shi, Fan Yang, Ze Wang, Xiaoxu Wu, Muzhi Guan, Guogang Liao, Wang Yongkang, Xingxing Wang, and Dong Wang. 2023. PIER: Permutation-Level Interest-Based End-to-End Re-ranking Framework in E-commerce. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Long Beach, CA, USA) (KDD '23). Association for Computing Machinery, New York, NY, USA, 4823–4831. doi:10.1145/3580305.3599886
- [11] Shuli Wang, Xue Wei, Senjie Kou, Chi Wang, Wenshuai Chen, Qi Tang, Yinhua Zhu, Xiong Xiao, and Xingxing Wang. 2025. NLGR: Utilizing Neighbor Lists for Generative Rerank in Personalized Recommendation Systems. In *Companion Proceedings of the ACM on Web Conference 2025* (Sydney NSW, Australia) (WWW '25). Association for Computing Machinery, New York, NY, USA, 530–537. doi:10.1145/3701716.3715251
- [12] Ruitao Zhu, Yangsu Liu, Dagui Chen, Zhenjia Ma, Chufeng Shi, Zhenzhe Zheng, Jie Zhang, Jian Xu, Bo Zheng, and Fan Wu. 2025. Contextual Generative Auction with Permutation-level Externalities for Online Advertising. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1* (Toronto ON, Canada) (KDD '25). Association for Computing Machinery, New York, NY, USA, 2171–2181. doi:10.1145/3690624.3709313